

Week 3 Skill Document

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

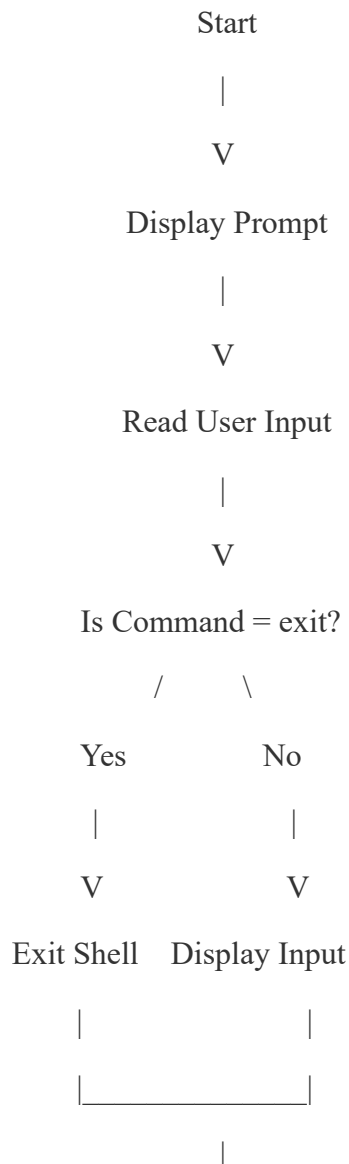
Description

The shell repeatedly displays a prompt, waits for user input, and exits when the user types **exit**.

Algorithm

1. Start the program.
2. Display the shell prompt.
3. Read the user input.
4. Check whether the entered command is **exit**.
5. If yes, terminate the shell.
6. Otherwise display the entered command.
7. Repeat the process.

Control Flow Diagram



V

End

```
speed@DESKTOP-P4G181S: ~ × Ubuntu × + v
wsl: Unknown key 'wsl2.autoMemoryReclaim' in C:\Users\speed\.wslconfig:3
speed@DESKTOP-P4G181S:~$ nano shell.c
speed@DESKTOP-P4G181S:~$ nano shell.c
speed@DESKTOP-P4G181S:~$ gcc shell.c -o shell
speed@DESKTOP-P4G181S:~$ ./shell
mysh> Hello
You entered: Hello
mysh> ls
You entered: ls
mysh> os
You entered: os
mysh> exit
Exiting shell...
speed@DESKTOP-P4G181S:~$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("mysh> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        printf("You entered: %s\n", command);
    }

    printf("Exiting shell...\n");

    return 0;
}
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: open(" filename.txt", modes, permissons)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    char buffer[100];
```

```
    int n;
```

```
    printf("Enter some text: ");
```

```
    n = read(0, buffer, sizeof(buffer) - 1);
```

```
    buffer[n] = '\0'; // Null-terminate the string
```

```
    printf("You entered: %s", buffer);
```

```
    return 0;
```

```
}
```

write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    write(1, "Hello World\n", 12);
```

```
    return 0;
```

```
}
```

Algorithm

1. Open **File1.txt** in read mode.
2. Open **File2.txt** in write mode.
3. Read data from File1.
4. Write data into File2.
5. Repeat until the end of the file.
6. Close both files.
7. Display a success message.

Working Flow

Start

|

Open File1

|

Open File2

|

Read Data

|

Write Data

|

More Data?

/ \

Yes No

| |

| Close Files

| |

-----|

|

Print Success

|

End

```
speed@DESKTOP-P4G181S: ~ × Ubuntu × +
speed@DESKTOP-P4G181S:~$ nano File1.txt
speed@DESKTOP-P4G181S:~$ nano File1.txt
speed@DESKTOP-P4G181S:~$ nano filecopy.c
speed@DESKTOP-P4G181S:~$ gcc filecopy.c -o filecopy
speed@DESKTOP-P4G181S:~$ ./filecopy
File copied successfully.
speed@DESKTOP-P4G181S:~$ cat File2.txt
Operating System and System Programming
Week 3 Skill Assignment
KL University
speed@DESKTOP-P4G181S:~$ █
```

GNU nano 8.7.1

Operating System and System Programming
Week 3 Skill Assignment
KL University

```
speed@DESKTOP-P4G181S: ~ × Ubuntu × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int src, dest;
    char buffer[1024];
    int bytes;

    src = open("File1.txt", O_RDONLY);

    if (src < 0)
    {
        printf("Cannot open File1.txt\n");
        return 1;
    }

    dest = open("File2.txt",
               O_WRONLY | O_CREAT | O_TRUNC,
               0644);

    while ((bytes = read(src, buffer, sizeof(buffer))) > 0)
    {
        write(dest, buffer, bytes);
    }

    close(src);
    close(dest);

    printf("File copied successfully.\n");

    return 0;
}
```